BRAC UNIVERSITY

Department of Computer Science and Engineering

Examination: Midterm
Duration: 80 Minutes
No. of Questions: 4

Semester: Fall 2025
Full Marks: 30
No. of Pages: 3

CSE 220: Data Structures

Name:	ID:	Section:

A

- ✔ Use the back **part** of the answer script for rough work. **No washroom breaks.**
- ✔ At the end of the exam, put the question **paper** inside the answer script and **return both**.
- ✔ **Understanding questions is part of the exam.**

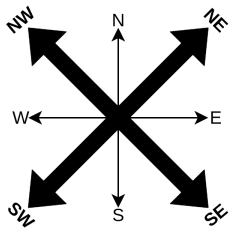
Question 1: CO5 [8 Points]

You are exploring an **energy grid matrix** using a **direction map matrix**, both of which are of an **m x n** shape.

- **grid[x][y]** – contains **integer** energy values where x , y are the indices.
- **directions[x][y]** – contains **String** direction symbols: **"NE", "NW", "SE", "SW"**.

Each cell of the **directions matrix** tells you in which direction the energy comes to that specific coordinate of the **grid matrix**. Meaning, for the sample example below the (0,1) cell is getting energy from all the cells which are "South East" of itself where the values are 7,12. So, the (0,1) cell's energy is 2+7+12=21.

Your task is to compute how much total energy each block absorbs by following the direction shown on the direction map. So, write a function/method **blockEnergySum()** that computes **a new Energy Matrix** where:



- For each cell in the grid matrix, start from its own position.
- Look at its direction symbol (i.e "SE" for [0,1]) in the directions matrix. Keep moving in that diagonal direction in the grid matrix until you exit the grid.
- While moving, you sum up all visited cell values, including the starting cell of each cell.
- Do this for all the cells of the grid matrix. Lastly, **return the newly computed Energy Matrix** where each of the cells contain the energy summations of each respective cell of the grid matrix.

Note: The space complexity of your solution must not exceed **O(m*n)**.

Given Matrices

grid =

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

directions =

"SE"	"SE"	"SW"	"NE"
"SW"	"SW"	"NE"	"SE"
"NW"	"SW"	"NE"	"NW"
"SE"	"NW"	"SW"	"NE"

Returned Energy Matrix

34	21	18	4
5	15	11	8
9	23	19	21
13	23	15	16

Explanation

```
[0,0]⇒ SE: includes its own value 1 then SE path adds 6,11,16 ⇒ 1+(6+11+16) = 34
[0,1]⇒ SE: includes its own value 2 then SE path adds 7,12 ⇒ 2+(7+12) = 21
[0,2]⇒ SW: includes its own value 3 then SW path adds 6,9 ⇒ 3+(6+9) = 18
[0,3]⇒ NE: includes its own value 4 but moving NE immediately exits the grid, so no additional cells
are
          added ⇒ total energy = 4 which is shown in the corresponding index of returned Energy Matrix.
[1,0]⇒ SW: same as [0,3] since the "SW" direction also exits the grid ⇒ total energy = 5.
[1,1]⇒ SW: includes its own value 6, then SW path adds 9 ⇒ 6+(9) = 15.
Other cells follow the same process: summing the cell's own value & cells along its diagonal direction.
```

Java Method Signature

```
static int[][] blockEnergySum( int[][] grid, String[][] directions ){
```

Python Function Signature

```
def blockEnergySum( grid , directions )
```

//TODO }	#TODO

Question 2: CO1 [8 Points]

Write a function/method named **rangeReverse()** that takes the **head** of a **dummy-headed singly linked list** and **two integers** as parameters. The two integers represent a **starting index** and an **ending index**.
You need to reverse the nodes within the given range. Furthermore, the nodes that lie outside the given range must remain in their original order and be appended after the reversed segment. Please note that this is an in-place operation meaning **you cannot create any new nodes.**

No need to write Node class. Assume, the **Node** class is already defined and contains: int elem and Node next.

You cannot create any new nodes to solve this task.

Given Linked List	Modified Linked List	Explanation
[DH]⇒*3*⇒*5*⇒**8**⇒**7**⇒**2**⇒**1**⇒*6*⇒*4*⇒*2* start = 2, end = 5	[DH]⇒**1**⇒**2**⇒**7**⇒**8**⇒*3*⇒*5*⇒*6*⇒*4*⇒*2*	Nodes from indices 2 to 5 were reversed and moved to the start. All other nodes remained in place and were appended after the reversed part.
[DH]⇒*3*⇒*5*⇒*8*⇒*7*⇒**2**⇒**1**⇒**6**⇒**4**⇒**5** start = 4, end = 12	[DH]⇒**5**⇒**4**⇒**6**⇒**1**⇒**2**⇒*3*⇒*5*⇒*8*⇒*7*	Nodes from indices 4 to 8 were reversed and moved to the start. All other nodes remained in place and were appended afterward. The end index 12 was ignored as it was outside the valid range.

Java Method Signature	Python Function Signature

static void rangeReverse(Node dHead, int start, int end){
// TODO
}
	def rangeReverse(dHead, start, end)
	#TODO

Question 3: CO1 [6 Points]

You are given two stacks, each sorted in **non-decreasing order from top to bottom**. Your task is to implement a function named **mergeStacks()** that merges the two stacks into a single stack and finally returns the merged stack. The merged stack also must be in non-decreasing order.

No need to write the Stack class. Assume, a linked list based stack is already implemented for you. **Its public instance methods are push(), pop(), peek(), and isEmpty()**. The instance variables of this class are **private**.

You cannot use any data structure other than the stack. You can create as many stacks as you want.

Given Stacks	Returned Stack	Explanation
stack1 = -2 3 6 6 10 stack2 = 0 4 6 8	stack = -2 0 3 4 6 6 6 8 10	Both stack1 and stack2 are in non-decreasing order. They are merged into a stack, and the merged stack is also in non-decreasing order.
Here, the leftmost value is the top		

Java Stack Creation	Python Stack Creation
Stack stk = new Stack();	stk = Stack()
Java Method Signature	**Python Method Signature**

static Stack mergeStacks(Stack stack1, Stack stack2) {
// TODO
}
	def mergeStacks(stack1, stack2):
	# TODO

Question 4: CO3 [8 Points]

Answer the questions in your Answer Script

i. Suppose you are given a multi-dimensional array with dimensions 6x4x8.
What is the linear index of [4][2][5] multidimensional index? [2 points]

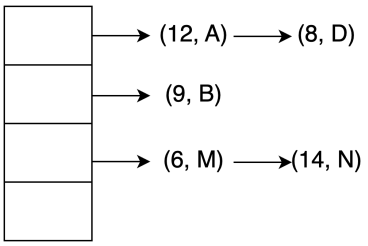
ii. In a linked list based queue, what is the time complexity of getting the first inserted element? [1 point]

a. O(1)
b. O(n)
c. O(log(n))
d. $O(n^2)$

iii. In a NxN 2D array, what will be the time complexity if we want to calculate the sum of the elements of the left diagonal in the most efficient way? [1 point]

a. O(log(n))
b. O(2n)
c. O(n)
d. $O(n^2)$

iv. For the hash table shown on the right, what can be a possible sequence in which the key value pairs are inserted? Here, the integer values are keys. Collision is resolved by appending a new item at the end.
[Hash Function = key%hash table length] [2 points]



a. (8, D), (12, A), (9, B), (6, M), (14, N)
b. (12, A), (6, M), (14, N), (8, D), (9, B)
c. (14, N), (9, B), (6, M), (8, D), (12, A)
d. (14, N), (9, B), (6, M), (12, A), (8, D)

v. Your two friends suggest the following hash functions to map **integer keys** into a hash table of length ***n***:

- friend 1's hash function = **key*2 % n**
- friend 2's hash function = **key % (n/2)**

Being a student who learned HashTable data structure, evaluate the 2 functions and find out:

a. What is the potential problem (if any) with Friend 1's hash function? [1 point]
b. What is the potential problem (if any) with Friend 2's hash function? [1 point]
Provide a one-line answer for each of the two questions above.